Vector Store Selection Strategy for Real Estate AI Platform

Objective

The vector store is a critical component of the retrieval architecture.

Its responsibility is to:

- Store embeddings efficiently.
- Support semantic and hybrid retrieval.
- Provide strong metadata filtering.
- Enable multi-tenancy.
- Scale to hundreds of millions of vectors.
- Support incremental updates.
- Maintain low latency and high recall.

The selection should be driven by requirements rather than popularity.

Requirements

Scale

Expected size:

- Millions of documents
- Tens or hundreds of millions of vectors

Growth:

- ~10K document updates/day
- Continuous ingestion

Multi-Tenancy

Multiple brokerage houses share the platform.

Requirements:

- Strong tenant isolation

- Efficient metadata filtering
- Flexible migration strategy

Hybrid Search

Support:

- Dense Vector Search
- Sparse/Keyword Search
- Metadata Filtering

Example:

"East-facing apartment near Whitefield metro under 1 crore"

Requires:

- Semantic understanding
- Structured filtering

Low Latency

Target:

P95 < 100 ms

Incremental Updates

Need to support:

- Document modifications
- Version changes
- Partial reindexing

Filtering

Examples:

Price:

80L–1Cr

Location:

Whitefield

Property Type:

Apartment

Tenant ID:

Broker_123

Filtering should happen during ANN traversal rather than after retrieval.

High Recall

Support:

- HNSW
- IVF
- Product Quantization

Evaluation Criteria

Search Quality

Metrics:

- Recall@K
- Precision@K
- Latency

Metadata Filtering

Should efficiently support:

- Tenant ID
- Price Range
- Property Type
- Property Status
- City

Multi-Tenancy

Should support:

- Shared collection
- Namespaces
- Partitions
- Dedicated collections

Scalability

Should support:

- 100M+ vectors
- Horizontal scaling

Incremental Updates

Need:

- Upsert
- Delete
- Versioning

Operational Complexity

Questions:

- Self-hosted?
- Managed?
- GPU required?
- Backup support?

Candidate Vector Stores

Qdrant

Strengths

- HNSW-based
- Excellent metadata filtering
- Payload-aware search
- Sparse + Dense support
- Quantization support
- Multi-vector support
- Incremental updates

Metadata filtering is integrated into graph traversal.

No expensive post-filtering step.

Very useful for:

- Tenant filtering
- Price filtering
- Location filtering

Scaling:

- Collections
- Shards
- Replication

Advantages

- Low latency
- Strong recall
- Cost-effective
- Open source

Limitations

- Requires operational management if self-hosted.

Pinecone

Strengths

- Fully managed
- Good scalability

- Namespace support
- Hybrid retrieval

Advantages

- Minimal operations

Limitations

- Expensive at scale
- Vendor lock-in
- Less flexibility

Vertex AI Vector Search

Strengths

- Managed service
- Deep GCP integration
- IVF support
- High scalability

Advantages

- Operational simplicity

Limitations

- Less flexible metadata filtering
- Vendor dependency

Weaviate

Strengths

- Hybrid search
- GraphQL support
- Multi-modal support

Advantages

- Rich ecosystem

Limitations

- Higher memory footprint

Milvus

Strengths

- HNSW
- IVF
- PQ
- GPU acceleration

Advantages

- Massive scale

Limitations

- Operational complexity
- Larger infrastructure footprint

Multi-Tenancy Strategy

Tenant isolation should evolve gradually.

Stage 1: Shared Collection

Store all tenants together.

Isolation via metadata:

tenant_id

Advantages:

- Lowest cost
- High utilization

Stage 2: Namespace / Partition

Large tenants receive dedicated partitions.

Benefits:

- Lower latency
- Reduced interference

Stage 3: Dedicated Collection

Large enterprise customers receive separate collections.

Benefits:

- Better isolation
- Independent scaling

Stage 4: Dedicated Infrastructure

Very large brokerage houses receive:

- Dedicated Vector Cluster
- Dedicated PostgreSQL
- Dedicated Cache

Maximum isolation.

Migration Strategy

Routing Layer

↓

Old Collection

↓

New Collection

Dual Writes

↓

Traffic Gradually Shifted

↓

100% Migration

↓

Retire Old Collection

No downtime migration.

Storage Strategy

Only child chunks are embedded.

Parent sections remain in:

- Document Store
- Object Store

Metadata stored with vectors:

- tenant_id
- document_id
- chunk_id
- parent_id
- property_id
- section
- version
- timestamp

This enables:

- Metadata filtering
- Parent expansion
- Version control

Indexing Strategy

Default:

HNSW

Advantages:

- High recall
- Low latency

For very large collections:

Combine:

IVF + PQ

Benefits:

- Memory reduction
- Faster search

Quantization:

- Scalar Quantization
- Product Quantization

Useful when vector count reaches hundreds of millions.

Hybrid Retrieval

Support:

Dense Search

+

Sparse Search

+

Metadata Filters

Example:

Query:

"Beautiful east-facing apartment with large rooms under 1 crore"

Dense Search:

Captures semantics.

Sparse Search:

Captures keywords.

Metadata Filter:

Price range Location Tenant

Combine results using:

Reciprocal Rank Fusion (RRF)

Then pass to reranker.

Incremental Update Strategy

When a document changes:

Avoid:

Re-index entire corpus

Instead:

Changed Section

↓

Chunk Detection

↓

Generate Embeddings

↓

Upsert Vector

↓

Delete Old Version

↓

Refresh Index

Minimizes cost.

Observability

Monitor:

Vector Search Latency

P50

P95

P99

Metadata Filter Latency

Index Size

Memory Usage

HNSW Graph Growth

Recall@K

Embedding Drift

Tenant Distribution

Cost Per Tenant

Recommendation

For this use case, I would recommend Qdrant.

Reasons:

1. Excellent metadata filtering integrated into HNSW traversal.
2. Strong support for multi-tenancy.
3. Supports dense + sparse + hybrid search.
4. Incremental updates are efficient.
5. Quantization support reduces memory footprint.
6. Scales to hundreds of millions of vectors.
7. Open source and cost-effective.
8. Flexible migration path:

Shared Collection

↓

Namespace

↓

Partition

↓

Dedicated Collection

↓

Dedicated Infrastructure

This allows balancing:

- Cost
- Latency
- Recall
- Tenant Isolation
- Scalability

while supporting future growth of large brokerage customers.

Final Recommendation

I would not optimize for today's scale.

I would optimize for evolutionary scalability.

Start with:

Shared Collection + Metadata Filtering

Scale to:

Partition per Tenant

Eventually move to:

Dedicated Collections and Infrastructure

This approach provides:

- Strong tenant isolation
- Cost efficiency
- Operational simplicity
- Low latency
- High recall
- Smooth migration path

without over-engineering the platform on day one.